AZ-104

AZ-104 Microsoft Azure Administrator

Edition 1 · Skills measured as of 2026-04-17

Source: https://learn.microsoft.com/en-us/credentials/certifications/resources/study-guides/az-104

Built 2026-11-01. An independent study aid, not Microsoft material.

Contents

AZ-104: Microsoft Azure Administrator 3

Scope and weights 3

How to use this book 3

Edition 3

1. Identities and governance (20–25%) 4

1.1 Microsoft Entra users and groups 4

1.2 Access to Azure resources with RBAC 6

1.3 Subscriptions, governance, and cost control 8

2. Storage (15–20%) 10

2.1 Storage access and network controls 10

2.2 Storage accounts and data operations 12

2.3 Azure Files and Blob Storage 14

3. Compute (20–25%) 17

3.1 Resource deployment with ARM and Bicep 17

3.2 Virtual machines and scale sets 19

3.3 Azure container services 21

3.4 Azure App Service 23

4. Virtual networking (15–20%) 25

4.1 Virtual networks, subnets, and routing 25

4.2 Secure access and private connectivity 27

4.3 Name resolution and load balancing 29

5. Monitoring and recovery (10–15%) 31

5.1 Azure Monitor and Network Watcher 31

5.2 Backup and disaster recovery 33

AZ-104: Microsoft Azure Administrator

An original study book for the AZ-104 certification exam. It follows the Microsoft AZ-104 study guide, skills measured as of April 17, 2026. It is an independent study aid, not Microsoft material.

Scope and weights

Chapter	Exam weight
1. Identities and governance	20–25%
2. Storage	15–20%
3. Compute	20–25%
4. Virtual networking	15–20%
5. Monitoring and recovery	10–15%

Source and interpretation notes: CURRICULUM.md. Objective-to-section map: coverage.md. The map supports review; it is not a guarantee of exhaustive coverage.

How to use this book

Before you start, read EXAM-FORMAT.md for how the AZ-104 exam is delivered and how to approach each question type, and use LABS.md for official hands-on labs and learning paths.

Use the coverage map to locate a skill, then study the matching topic. Each topic explains the learning intent, core concepts, important distinctions and limits, a worked operational scenario, a practical activity, exam application, and self-check questions with answers. For hands-on activities, use a non-production subscription and check current pricing, permissions, quotas, and cleanup steps before creating resources.

AZ-104 tests administration in context: identify the target resource and scope, choose a least-privilege control, account for dependencies and network boundaries, verify effective configuration, and confirm the result. Do not rely on a remembered portal layout alone; use the linked documentation to verify version-sensitive behavior.

Edition

Edition 1 (2026-10-12), based on the April 17, 2026 skills outline. Content is drafted and has not yet been human-reviewed. See CHANGELOG.md.

CHAPTER 1

Identities and governance (20–25%)

Azure administration begins with two questions: **who is acting, and which resources may they change?** Governance adds the rules that keep those changes within an organization's boundaries. These capabilities operate at different layers: Microsoft Entra ID holds directory identities, Azure role-based access control (Azure RBAC) authorizes actions on Azure resources, and Azure Policy evaluates resource configuration against rules.

1.1 Microsoft Entra users and groups

Learning intent

Create and manage directory users and groups, understand licenses and external identities, and configure self-service password reset (SSPR) with the required prerequisites.

Core concepts

A **user** is a directory identity. User properties such as display name, sign-in name, account state, and assigned directory roles have different purposes; changing a profile property does not by itself grant access to an Azure subscription. A **group** collects identities so membership and access can be managed consistently. Security groups are commonly used for access control; Microsoft 365 groups also support collaboration workloads. Membership can be assigned by an administrator or, where supported and licensed, evaluated dynamically from user attributes.

Product licenses enable particular capabilities; they are not Azure resource permissions. Assign a license to the intended users or groups, and verify the current service-plan and licensing requirements before enabling a feature. An **external user** is an identity from outside the organization's workforce directory that is invited or otherwise brought into collaboration. External collaboration settings and resource authorization are separate controls: invitation does not automatically grant access to every resource.

SSPR lets eligible users reset their own password after the organization configures the feature, registration and authentication methods, and applicable scope. It is not a substitute for administrator recovery procedures or strong authentication. Requirements vary with directory configuration and licensing.

How it works, distinctions, limits, and defaults

Treat the directory and the Azure resource hierarchy as related but distinct systems. A user can exist in Entra ID without being assigned an Azure role. Group-based administration can reduce repeated assignments, but membership lifecycle and nested-group behavior must be checked for the consuming service. Minimize standing privileged access and review group owners and members.

Example or worked scenario

A company hires 30 contractors for a migration. An administrator provisions named identities, places them in a contractor security group, grants that group a narrowly scoped role on the migration resource group, and configures the approved sign-in and external-collaboration requirements. The identity lifecycle team sets an end date and removes membership when the contract ends. The group assignment is easier to review than 30 independent resource grants.

Hands-on activity

In a test tenant, create a nonprivileged test user and security group, inspect their properties and membership, then assign and remove a test license if available. Review the Entra external collaboration and SSPR configuration pages without changing production settings. Record which steps concern identity, licensing, or resource authorization.

Official labs

- Lab 01 – Manage Microsoft Entra ID identities
- Learning path: AZ-104: Manage identities and governance in Azure

Labs run in your own Azure subscription and may create billable resources; see LABS.md and delete resources when finished.

Exam or practical application

When a scenario asks who can sign in, look at identity and authentication configuration. When it asks whether a person can manage a resource, inspect Azure RBAC assignments. A guest invitation, group membership, license, and role assignment solve different problems.

Self-check questions

1. Does adding a user to the Entra directory grant Contributor access to a subscription?

   Answer: No. An Azure role assignment at an applicable scope is needed.

2. What is the administrative benefit of assigning a role to a group?

   Answer: Permissions can be granted once and then managed through group membership, making access review and lifecycle changes more consistent.

3. Does inviting an external user automatically authorize that user to manage Azure resources?

   Answer: No. External identity access and Azure resource authorization are separate.

Sources

- Microsoft Entra ID documentation
- Create or delete users in Microsoft Entra ID
- Manage groups in Microsoft Entra ID
- Assign or unassign licenses for users in the Microsoft 365 admin center

- What is Microsoft Entra External ID?
- Self-service password reset

1.2 Access to Azure resources with RBAC

Learning intent

Select appropriate built-in roles, assign them at a suitable scope, and interpret effective role assignments without confusing directory roles with Azure resource roles.

Core concepts

An Azure RBAC role definition is a set of allowed actions. A **role assignment** connects a security principal (user, group, service principal, or managed identity) to a role at a **scope**. The main scopes form an inheritance hierarchy: management group, subscription, resource group, and individual resource. An assignment at a parent scope normally applies to descendants unless the role definition or assignment conditions constrain it.

Built-in roles represent common permission sets. **Owner** can manage resources and access; **Contributor** can manage resources but cannot grant access; **Reader** can view resources. Service-specific roles, such as Storage Blob Data Reader, target particular data-plane operations. These names are useful orientation, not a reason to grant broad access. Choose the least-privileged role that permits the required task, and check its current actions and data actions in the role definition.

How it works, distinctions, limits, and defaults

Azure management-plane permissions control operations on resource configuration; data-plane permissions control operations on data stored in a service. A management role does not necessarily imply data access. Effective authorization can be affected by inherited assignments, deny assignments, conditions, resource-specific behavior, and directory or service configuration. A role assignment is therefore not always the whole access story.

Scope is a security boundary and an operational choice. Subscription scope is convenient for cross-resource work but grants broad reach; a resource or resource-group scope limits blast radius. Role assignments can take time to propagate. Removing an assignment may not immediately terminate existing sessions or cached credentials.

Example or worked scenario

A support engineer must restart one virtual machine but must not change its network or grant access. First check whether an appropriate built-in VM operator role supplies the necessary actions. Assign it to a support group at the VM or tightly scoped resource-group level, then test the permitted operation and verify that unrelated actions remain denied.

Hands-on activity

In a disposable resource group, inspect **Access control (IAM)** and list role assignments inherited from the subscription and management group. Compare Reader and Contributor role definitions. If a test identity is available, assign the narrowest suitable role, verify an allowed action, then remove the assignment.

Official labs

- Lab 02a – Manage subscriptions and RBAC
- Learning path: AZ-104: Manage identities and governance in Azure

Labs run in your own Azure subscription and may create billable resources; see LABS.md and delete resources when finished.

Exam or practical application

Read a scenario as a tuple: **principal + required operation + target + duration**. Match the operation to a role, then choose the lowest scope that includes the target resources. Do not mistake an Entra directory role for an Azure RBAC role.

Self-check questions

1. Which common role allows resource management but not the ability to grant other users access: Owner or Contributor?

   Answer: Contributor.

2. If a user has Reader on a subscription, does assigning Contributor on one resource group remove the inherited Reader access elsewhere?

   Answer: No. The resource-group assignment adds permissions within its scope; the inherited subscription assignment still applies.

3. Why might a management-plane role not allow reading a blob's contents?

   Answer: Blob reads are data-plane operations and may require a data role or another service-specific authorization path.

Sources

- What is Azure role-based access control (Azure RBAC)?
- Understand scope for Azure RBAC
- Built-in Azure roles
- Assign Azure roles using the Azure portal
- Azure RBAC and Azure attribute-based access control (ABAC)

1.3 Subscriptions, governance, and cost control

Learning intent

Organize resources and subscriptions, apply governance consistently with policy and locks, use tags thoughtfully, and use budgets, alerts, and Azure Advisor to manage costs.

Core concepts

A **resource group** is a management container for resources that are commonly deployed, updated, and retired together. A resource belongs to one resource group at a time, and the group has a location for its metadata; contained resources can be in other regions. **Subscriptions** provide an administrative and billing boundary for Azure resources. **Management groups** let administrators organize subscriptions and apply governance across them. Together these scopes form a hierarchy; a policy or role assignment at a parent scope can affect descendants.

Azure Policy evaluates resources against definitions and assignments. Depending on its effect, a policy can audit, deny, append or modify configuration, or deploy related configuration when remediation is appropriate. An audit result reports noncompliance; it does not automatically correct a resource. Remediation tasks and permissions may be required for effects that modify existing resources. **Resource locks** provide an additional guard against deletion or changes: a Delete lock blocks deletion while allowing updates; a ReadOnly lock blocks writes. Locks do not replace access control and must be managed deliberately.

Tags are name/value metadata useful for ownership, environment, cost allocation, and automation. Tags do not automatically inherit from a resource group or subscription to its resources; policy can enforce or add tags. Cost Management supports analysis, budgets, and alerts. A budget is a monitoring and notification mechanism, not a hard spending cap. **Azure Advisor** provides recommendations across areas such as cost, reliability, security, operational excellence, and performance; recommendations need validation against the workload.

How it works, distinctions, limits, and defaults

Use resource groups for lifecycle and administration, subscriptions for boundaries such as billing or environment separation, and management groups for cross-subscription governance. Moving resources between scopes can have service-specific restrictions and can affect dependencies or role assignments; inspect move guidance before proceeding.

Policy and locks answer different questions. Policy evaluates whether configuration meets a rule; a lock restricts certain management operations. Neither makes a poor role assignment safe. Cost alerts signal a threshold or forecast; they should be paired with owners, review cadence, and response actions.

Example or worked scenario

A team operates development and production subscriptions. A management-group policy audits required cost-center tags, while production receives stricter location and SKU rules. Resource groups follow application lifecycles. Budgets alert each owner before expected monthly spend is exceeded; they do not stop deployment, so the team also reviews forecasts and Advisor recommendations.

Hands-on activity

In a sandbox, create a resource group with an owner/environment tag, inspect policy compliance, and compare Delete and ReadOnly locks on a disposable resource. If the subscription allows it, create a low-value budget alert with a test contact. Remove temporary resources and review current costs before leaving the sandbox.

Official labs

- Lab 02a – Manage subscriptions and RBAC
- Lab 02b – Manage governance via Azure Policy
- Learning path: AZ-104: Manage identities and governance in Azure

Labs run in your own Azure subscription and may create billable resources; see LABS.md and delete resources when finished.

Exam or practical application

For a governance scenario, identify the scope where a rule should apply and whether the requirement is to report, deny, modify, or protect. For a cost question, distinguish a budget alert from a spending limit, and treat Advisor as a recommendation source rather than an enforcement engine.

Self-check questions

1. Will a tag on a resource group automatically appear on every resource in that group?

   Answer: No. Use policy or another management process to apply or maintain tags on resources.

2. Which control is designed to report resources that violate a configuration rule: Azure Policy or a resource lock?

   Answer: Azure Policy. A lock restricts certain management operations.

3. Does reaching an Azure budget automatically prevent further charges?

   Answer: No. Budgets provide monitoring and alerts; they do not hard-cap consumption.

Sources

- Azure resource manager overview
- Management groups
- Azure Policy overview
- Remediate noncompliant resources with Azure Policy
- Lock your Azure resources to protect your infrastructure
- Use tags to organize your Azure resources
- Create and manage budgets
- Azure Advisor

CHAPTER 2

Storage (15–20%)

Azure Storage is a set of services with distinct data models, protocols, and availability options. Administration means matching the service to the workload, limiting who and where can access it, and planning for the failures that protection features can and cannot recover from.

2.1 Storage access and network controls

Learning intent

Configure network boundaries and authorization for storage, including shared access signatures (SAS), stored access policies, account keys, and identity-based access for Azure Files.

Core concepts

Azure Storage authorization can use Microsoft Entra identities, shared keys, SAS tokens, and service-specific mechanisms. Prefer identity-based authorization where it fits: assign a data role to a principal at the narrowest suitable scope, and use network rules as an additional boundary. **Storage firewalls and virtual network rules** restrict requests to approved network paths; a permitted identity does not automatically bypass those network restrictions.

A **SAS** is a signed, time-limited delegation of specific permissions to storage resources. Scope, permissions, start/expiry times, and allowed protocols should be minimized. A service SAS can be associated with a **stored access policy** for supported services, allowing administrators to change or revoke the policy that governs its SAS tokens. Not all SAS types support stored access policies. An account SAS and a user delegation SAS have different authority and signing requirements; use a user delegation SAS for Blob Storage when identity-based delegation is appropriate.

Storage account keys grant broad access to the account. Keep them secret, avoid embedding them in source code, and rotate them using the account's two-key lifecycle where applications can be updated safely. For **Azure Files**, identity-based SMB authentication can integrate with supported directory services and permit share/file authorization through identities; support depends on protocol, client, directory configuration, and deployment scenario. A storage account key is not equivalent to per-user identity authorization.

How it works, distinctions, limits, and defaults

Authorization and network access are separate checks. A caller may have a valid role or SAS and still be blocked by firewall, routing, private endpoint, or public-network settings. Conversely, a reachable endpoint does not authorize data operations. SAS expiration policy, key rotation, logging, and secret handling should be part of operations rather than afterthoughts.

Private endpoints provide a private IP path to a storage service through a virtual network, but require correct DNS and routing. Service endpoints keep access on the Azure backbone and let a storage firewall allow selected subnets; they do not provide a private IP address for the service. Verify supported endpoint types and current data-exfiltration controls for the specific storage service.

Example or worked scenario

A reporting job needs read-only access to one container for several hours. Prefer a managed identity with a container-scoped data role if the job runs in Azure and its service supports identity authorization. If a temporary external process requires a SAS, restrict it to the needed resource and read permission, use HTTPS, set a short expiry, and protect the token as a credential. Restricting the network path adds another layer.

Hands-on activity

In a sandbox storage account, compare a role-based data assignment with a short-lived, read-only SAS. Test each from an allowed and disallowed network path, then revoke the assignment or token and review access configuration. Never paste live keys or SAS values into notes or source control.

Official labs

- Lab 07 – Manage Azure Storage
- Learning path: AZ-104: Implement and manage storage in Azure

Labs run in your own Azure subscription and may create billable resources; see LABS.md and delete resources when finished.

Exam or practical application

Determine whether a scenario needs resource access (role), a delegated temporary capability (SAS), broad account-level access (key), or SMB identity authorization (Azure Files). Then separately evaluate network restrictions, expiry, and revocation.

Self-check questions

1. Does allowing a subnet through a storage firewall grant that subnet's users permission to read blobs?

   Answer: No. The firewall controls network reachability; authorization is still required.

2. Which credential generally has broader authority: an account key or a narrowly scoped SAS?

   Answer: An account key generally grants broad account-level authority; a SAS can be restricted by resource, permission, and time.

3. Are stored access policies available for every type of SAS?

   Answer: No. They apply only to supported service SAS scenarios; check the current service documentation.

Sources

- Authorize access to blobs using Microsoft Entra ID
- Grant limited access to Azure Storage resources using SAS
- Define a stored access policy
- Manage storage account access keys
- Configure Azure Storage firewalls and virtual networks
- Azure Files identity-based authentication
- Choose an Azure Files identity-based authentication option

2.2 Storage accounts and data operations

Learning intent

Create and configure storage accounts, select redundancy and replication options, understand encryption choices, and move or manage data with Storage Explorer and AzCopy.

Core concepts

A **storage account** is the namespace and configuration boundary for supported Azure Storage services. Account configuration includes performance and redundancy choices, networking, security, and data-protection settings. Choose the account kind and performance tier that fit the required services and workload; options, features, and constraints vary by region and service.

Redundancy controls how copies are maintained. Locally redundant storage (LRS) keeps replicas within one datacenter in a region. Zone-redundant storage (ZRS) distributes copies across availability zones where supported. Geo-redundant options asynchronously replicate data to a paired secondary region; read-access variants provide a read endpoint for the secondary copy. These choices address different failure domains and recovery objectives. Replication is not the same as backup: deletion or corruption can be replicated.

Object replication asynchronously copies eligible blobs between source and destination accounts according to configured rules. It supports distribution or recovery scenarios but is not instantaneous and does not replace versioning, soft delete, or a backup plan. Storage encrypts data at rest by default; accounts can use Microsoft-managed keys or customer-managed keys where supported. Encryption at rest does not replace access control or secure transport.

Azure Storage Explorer provides a graphical way to inspect and manage supported storage resources. **AzCopy** is a command-line utility for data transfer and synchronization scenarios. Both still require authorization, correct endpoint/network access, and a transfer plan that accounts for scale, retries, and verification.

How it works, distinctions, limits, and defaults

Select redundancy by failure domain and recovery requirement, not by label alone. Geographic replication is asynchronous, so a regional failure can involve a recovery point other than the latest write. Not every account type or service supports every redundancy option, and changing redundancy can involve service or cost constraints. Check the current feature matrix.

Encryption protects stored data from unauthorized disclosure if underlying media are accessed; a principal with valid service authorization can still read data. Customer-managed keys add key-management responsibilities and dependencies. Treat transfer utilities as clients, not as authorization systems.

Example or worked scenario

A media team stores source images in a Blob account and needs copies in another region for downstream processing. It compares ZRS availability and geo-redundancy requirements, checks object-replication support and expected delay, and separately configures soft delete/versioning to recover from accidental overwrites. AzCopy handles an initial bulk transfer; a managed identity limits ongoing access.

Hands-on activity

Create a low-cost test account only after checking current pricing. Compare available redundancy, encryption, and network settings in the portal. Upload a small test file with Storage Explorer, then transfer it with AzCopy using an identity-based sign-in. Verify the destination and remove the test account when finished.

Official labs

- Lab 07 – Manage Azure Storage
- Learning path: AZ-104: Implement and manage storage in Azure

Labs run in your own Azure subscription and may create billable resources; see LABS.md and delete resources when finished.

Exam or practical application

For account configuration, map each requirement to a setting: failure-domain coverage, key ownership, access method, network boundary, and data movement. If recovery from deletion is required, do not assume replication alone is sufficient.

Self-check questions

1. What is the key difference between ZRS and geo-redundant storage?

   Answer: ZRS replicates across availability zones in a region; geo-redundancy asynchronously maintains a copy in a secondary region.

2. Does object replication necessarily copy a newly written blob immediately?

   Answer: No. It is asynchronous and replication time can vary.

3. Does encryption at rest prevent an authorized identity from reading a blob?

   Answer: No. Encryption at rest does not replace authorization controls.

Sources

- Storage account overview
- Azure Storage redundancy
- Object replication for block blobs
- Azure Storage encryption for data at rest
- Azure Storage Explorer
- Get started with AzCopy

2.3 Azure Files and Blob Storage

Learning intent

Create file shares and blob containers, select storage tiers, and configure protection and lifecycle features for Azure Files and Blob Storage.

Core concepts

Azure Files provides managed file shares accessible over supported protocols, commonly SMB and NFS depending on configuration. Create a share with a suitable quota and performance option, then configure identity, network, and client access. Share snapshots provide point-in-time views useful for recovery; soft delete retains deleted shares for a configured period. Neither feature guarantees recovery from every failure or replaces a separately designed backup.

Azure Blob Storage stores object data in containers. A container is an organizational and access-control boundary; blobs carry the data. Access tiers such as hot, cool, cold, and archive (where supported) trade access cost, storage cost, and retrieval behavior. Archive data must be rehydrated before normal reads; tier availability and minimum retention or early-deletion charges vary by account and tier. Do not move data to a colder tier solely on access frequency without considering retrieval needs.

Blob **soft delete** retains deleted blobs or containers for a configured retention period. **Versioning** keeps previous blob versions as writes occur. **Lifecycle management** applies rule-based tiering or deletion based on age and other supported conditions. Azure Files has its own snapshots and soft-delete capabilities; do not assume Blob protection settings apply to file shares. Protection settings can interact, and lifecycle rules may delete older versions if configured to do so.

How it works, distinctions, limits, and defaults

Soft delete provides a recovery window; versioning provides prior object states; snapshots provide point-in-time views. These are operational protections but may share account-level failure or administrative boundaries. Review retention, capacity, and recovery workflows. Test restoring representative content before relying on a feature.

Lifecycle management is asynchronous and conditions are evaluated over time, not necessarily at the exact moment a threshold is crossed. Some operations, tiers, account configurations, and protection features have mutual constraints. Verify current support and cost implications.

Example or worked scenario

A design team collaborates through an Azure file share, while a publishing system stores product images as blobs. The file share uses identity-based SMB access and snapshots for convenient file recovery. The Blob account enables versioning and soft delete for accidental edits, and lifecycle rules move older, rarely accessed images to a lower-cost tier. Legal retention requirements are handled separately from these operational settings.

Hands-on activity

In a sandbox, create a file share and a blob container, upload non-sensitive test data, and enable an available protection feature. Delete a test item and restore it using the documented workflow. Inspect tier and lifecycle options, then clean up the test data and resources.

Official labs

- Lab 07 – Manage Azure Storage
- Learning path: AZ-104: Implement and manage storage in Azure

Labs run in your own Azure subscription and may create billable resources; see LABS.md and delete resources when finished.

Exam or practical application

Map requirements to the right service and recovery mechanism: shared file-system access suggests Azure Files; object data suggests Blob Storage. Distinguish a share snapshot, blob version, soft-delete retention, and lifecycle action; each solves a different problem.

Self-check questions

1. Which service is designed for a managed file share: Azure Files or Blob Storage?

   Answer: Azure Files.

2. What is one important difference between blob versioning and lifecycle management?

   Answer: Versioning preserves prior blob states; lifecycle management applies rules that can tier or delete eligible data.

3. Can archived blob data always be read immediately like hot-tier data?

   Answer: No. Archive data must generally be rehydrated, and access behavior and costs differ.

Sources

- Introduction to Azure Files
- Create an Azure file share

- Overview of Azure Files share snapshots
- Azure Files soft delete
- Introduction to Azure Blob Storage
- Access tiers for blob data
- Soft delete for blobs
- Blob versioning
- Optimize costs by automating Azure Blob Storage access tiers

CHAPTER 3

Compute (20–25%)

Compute administration ranges from declarative resource deployment to maintaining individual virtual machines and managed application platforms. The operational choice is a balance among control, responsibility, scaling model, and service limits.

3.1 Resource deployment with ARM and Bicep

Learning intent

Read, modify, and deploy Azure Resource Manager (ARM) templates and Bicep files; understand parameters, dependencies, outputs, and how to export or convert deployments.

Core concepts

Azure Resource Manager (ARM) is the management plane for deploying and managing Azure resources. An ARM JSON template declares the target resource types, API versions, properties, parameters, variables, dependencies, and outputs. **Bicep** is a declarative language that compiles to ARM JSON and offers a more concise syntax while retaining ARM deployment behavior.

Templates make deployments repeatable. Parameters vary values between environments; variables hold reusable expressions; outputs expose selected deployment results. Dependencies express ordering when ARM cannot infer it from resource references. A deployment's target scope matters: templates can deploy at resource-group, subscription, management-group, or tenant scope when supported by the deployment type and permissions.

An exported template can help document or reproduce an existing deployment, but it may omit operational intent, secrets, or configuration not represented by the export. Converting an ARM template to Bicep is supported by tooling, but generated output should be reviewed and tested. Use source control and parameter files carefully; never commit secrets.

How it works, distinctions, limits, and defaults

Declarative deployment describes the desired resource state, not an imperative sequence for every action. A template deployment does not guarantee that all application-level setup is complete or that the resulting service is secure. Resource providers, API versions, permissions, quotas, and region availability affect success.

Review changes before deployment, especially replacements, destructive property changes, role assignments, and network exposure. Use incremental deployments where appropriate and understand the selected deployment mode and scope. Keep secrets in a managed secret store or secure deployment mechanism rather than template literals.

Example or worked scenario

A team needs identical test and production network foundations. It creates a Bicep template for a virtual network and subnets, parameterizes address ranges and environment tags, and uses separate parameter values for each environment. A deployment preview and code review catch an unintended subnet change before it reaches production.

Hands-on activity

In a test subscription, use the Azure portal or Azure CLI to validate and deploy a small Bicep template containing a resource group-scoped resource. Inspect deployment operations and outputs, then remove the test resource. Export a deployment and compare it with the original source; note any missing intent.

Official labs

- Lab 03 – Manage Azure resources by using ARM templates
- Learning path: AZ-104: Deploy and manage Azure compute resources

Labs run in your own Azure subscription and may create billable resources; see LABS.md and delete resources when finished.

Exam or practical application

When interpreting a template, identify its scope, parameters, resource type/API version, dependencies, and outputs. When asked to modify it, change the declarative property or parameter that represents the requirement, then validate the deployment and its permissions.

Self-check questions

1. What is the relationship between Bicep and ARM templates?

   Answer: Bicep is a declarative language that compiles to ARM JSON templates and uses ARM for deployment.

2. Why might a resource need an explicit dependency?

   Answer: To ensure deployment ordering when a dependency is not already implied by a resource reference.

3. Is an exported template always a complete and safe source of truth for redeployment?

   Answer: No. It may omit intent or configuration, can require review, and should not be assumed to include secrets or all operational details.

Sources

- Azure Resource Manager templates overview
- Bicep overview
- ARM template structure and syntax
- Deploy resources with Bicep and Azure CLI

- Export template in the Azure portal
- Decompile an ARM template to Bicep

3.2 Virtual machines and scale sets

Learning intent

Create and configure virtual machines (VMs), select sizes and disks, protect host data, move VMs between supported scopes, and plan availability with zones, availability sets, and scale sets.

Core concepts

An Azure VM is a compute resource with dependencies such as a virtual network interface, operating-system disk, optional data disks, and public or private IP configuration. Select an image, size, OS, authentication method, region, and availability design based on workload requirements. A VM **size** determines available CPU, memory, storage, and network characteristics; available sizes vary by region and quota. A managed disk abstracts the underlying storage account and offers selectable performance and redundancy characteristics.

Encryption at host encrypts VM-host caches and temporary disks in addition to managed disks where supported. It is a host-level protection option, not a replacement for disk encryption, key management, or access controls. Review supported VM sizes, regions, and dependencies before enabling it.

Availability zones place resources in separate physical locations within a supported region. **Availability sets** distribute VMs across fault and update domains within a datacenter. They are different placement approaches; neither makes an application highly available if its dependencies remain a single point of failure. **Virtual Machine Scale Sets (VMSS)** manage a group of VMs with coordinated deployment and scaling capabilities. Flexible and uniform orchestration modes have different behaviors; check the current design guidance.

Moving a VM across resource groups, subscriptions, or regions is subject to resource-specific move support and dependency constraints. A region move is often a redeployment or replication workflow rather than a simple metadata change. Check the move matrix and plan downtime, IP/DNS changes, quotas, and dependent resources.

How it works, distinctions, limits, and defaults

Disk performance and VM network throughput can be constrained by both the VM size and the disk tier. Changing a VM size can require deallocation and is limited by regional capacity. A VM's availability model must match application architecture: stateless replicas may scale horizontally, while stateful workloads need data replication and recovery planning.

Availability sets distribute placement within one datacenter; zones distribute across zone locations in a region. A scale set can support scale-out but does not automatically make an application resilient. Backup, patching, identity, and network security remain operational responsibilities.

Example or worked scenario

A line-of-business service requires OS-level control and must tolerate planned maintenance. The team selects a VM size after measuring memory and network needs, places redundant instances across zones where the region and application allow it, and stores state in a separately resilient service. It checks quotas and capacity, configures backup, and tests application failover instead of assuming VM placement alone provides availability.

Hands-on activity

In a sandbox, review VM sizes and disk SKUs without deploying. If authorized, deploy a small test VM with a managed identity and no unnecessary public endpoint; inspect its disk and availability configuration. Use a move validation operation before any cross-scope move, and delete the VM and disks after testing.

Official labs

- Lab 08 – Manage virtual machines
- Learning path: AZ-104: Deploy and manage Azure compute resources

Labs run in your own Azure subscription and may create billable resources; see LABS.md and delete resources when finished.

Exam or practical application

Start with the requirement: capacity points to size, persistent storage to managed disks, physical-location failure to zones, planned maintenance within a datacenter to availability sets, and group lifecycle or autoscaling to VMSS. Verify that the region, SKU, quota, and dependencies support the requested design.

Self-check questions

1. How do availability zones differ from availability sets?

   Answer: Zones are separate physical locations in a region; availability sets distribute VMs across fault and update domains within a datacenter.

2. Does a larger VM size guarantee better performance for every workload?

   Answer: No. The selected size must fit the workload, and storage, network, application, quota, and regional capacity can also constrain performance.

3. Is moving a VM to another region always a direct resource move?

   Answer: No. Regional moves often require a supported migration or redeployment approach and planning for dependencies and downtime.

Sources

- Sizes for virtual machines in Azure
- Managed disks overview
- Encryption at host

- Availability options for virtual machines in Azure
- Availability sets overview
- Availability zones overview
- Virtual Machine Scale Sets overview
- Move operation support for resources
- Move Azure VMs to another region

3.3 Azure container services

Learning intent

Create and manage an Azure Container Registry (ACR), choose between Azure Container Instances (ACI) and Azure Container Apps for container workloads, and configure sizing and scaling appropriate to each.

Core concepts

A **container image** packages an application and its dependencies. **Azure Container Registry** stores and distributes images. Configure registry access through appropriate identities and role assignments, use private networking where needed, and protect image supply chains with controlled builds and updates. An image registry stores artifacts; it does not run the container.

Azure Container Instances runs container groups without requiring VM or orchestrator administration. It fits simple, short-lived, or isolated container workloads. **Azure Container Apps** is a managed application platform for containerized services, with features for revisions, ingress, and event-driven scaling. Neither should be confused with managing a full Kubernetes cluster; choose a more orchestration-focused platform only when its capabilities are actually needed.

Container sizing concerns CPU and memory requests/limits and the workload's scale model. ACI container groups have resource and lifecycle characteristics that differ from Container Apps. Container Apps can scale based on supported triggers and configuration, but scale-to-zero behavior, replicas, concurrency, and minimum/maximum replicas depend on the app and rule settings.

How it works, distinctions, limits, and defaults

Registry authentication, image pull permissions, ingress, secrets, and egress must all be configured intentionally. A container image can include vulnerabilities or secrets; scanning and controlled build pipelines are part of administration. Scaling compute does not necessarily scale a backing database or external dependency.

ACI is not a general-purpose orchestration layer; Container Apps supplies more application-level management. Service quotas, supported regions, networking features, and billing dimensions change, so check current service limits before sizing a production workload.

Example or worked scenario

A scheduled conversion job runs for a few minutes and exits; ACI may suit its simple lifecycle. A customer-facing API needs revisions, ingress, and event-driven horizontal scaling; Container Apps may better match. Both pull images from a controlled registry, use managed identity where supported, and receive only required network access.

Hands-on activity

Inspect an existing ACR's repositories, access configuration, and image tags. In a sandbox, deploy a simple non-sensitive test image to ACI or Container Apps, review logs, ingress, resource sizing, and scaling configuration, then delete the workload and any unused image.

Official labs

- Lab 09b – Implement Azure Container Instances
- Lab 09c – Implement Azure Container Apps
- Learning path: AZ-104: Deploy and manage Azure compute resources

Labs run in your own Azure subscription and may create billable resources; see LABS.md and delete resources when finished.

Exam or practical application

First identify the registry requirement separately from runtime. Then compare the requested lifecycle and operational features: simple container execution suggests ACI; managed app revisions and configurable scale/ingress suggest Container Apps. Always include the image pull identity and network path.

Self-check questions

1. Does ACR execute a container image?

   Answer: No. It stores and distributes images; a runtime such as ACI or Container Apps executes them.

2. Which service is more aligned with a simple isolated container group: ACI or Azure Container Apps?

   Answer: ACI.

3. If a Container App scales out, is its database guaranteed to scale with it?

   Answer: No. The database is a separate dependency with its own limits and scaling configuration.

Sources

- Azure Container Registry overview
- Azure Container Instances overview
- Azure Container Apps overview
- Set scaling rules in Azure Container Apps
- Azure Container Apps revisions

3.4 Azure App Service

Learning intent

Provision an App Service plan and web app; configure scaling, certificates and TLS, custom domains, backups, networking, and deployment slots.

Core concepts

Azure App Service hosts web applications and APIs on a managed platform. An **App Service plan** defines the compute resources, operating system, region, pricing tier, and scale capacity for one or more apps. Apps in the same plan share the plan's compute resources; scaling the plan affects its apps. Choose a tier based on required features, capacity, availability, and cost.

Scale up changes the plan's pricing tier or instance size; **scale out** changes the number of instances. Autoscale options and supported behavior depend on the plan tier and configuration. A custom domain requires domain ownership verification and appropriate DNS records. Configure TLS binding and certificates for HTTPS; certificate availability and renewal method depend on the chosen certificate and domain.

Deployment slots provide separate live app environments, commonly used to validate a release before swapping into production. Slot settings and app configuration must be reviewed because selected settings can remain bound to a slot while others move during a swap. **App Service backups** are configured with supported storage and schedule/retention options depending on plan and app configuration. Backups should be tested, monitored, and protected from unauthorized access.

Networking features include inbound access restrictions, virtual network integration for supported outbound connectivity, and private endpoints for private inbound access. These features have different directions and DNS requirements: an outbound integration path is not the same as a private endpoint to the app.

How it works, distinctions, limits, and defaults

The plan is the compute boundary; the app is the workload configuration boundary. Apps sharing a plan can contend for resources and share cost. Scaling an app's plan may affect other apps on it. TLS protects transport only when the hostname, certificate, and binding are correct. DNS changes can take time to propagate.

Slots reduce deployment risk but do not replace application testing, database migration strategy, or rollback planning. Backup availability and content can depend on plan tier, app type, linked storage, and current platform behavior; verify requirements before relying on it.

Example or worked scenario

A team operates an API with a custom hostname. It provisions a plan sized for expected traffic, binds a verified domain and TLS certificate, and uses a staging slot to validate releases. Inbound restrictions limit access, while VNet integration provides a path to a private backend. Before swapping, the team confirms slot-specific settings and verifies that backup recovery covers the app's configuration and content needs.

Hands-on activity

In a test subscription, compare App Service plan tiers and inspect scale settings. Deploy a sample app, create a staging slot if supported, and review domain, TLS, networking, and backup blades without exposing a production app. Delete the plan and test resources afterward.

Official labs

- Lab 09a – Implement web apps
- Learning path: AZ-104: Deploy and manage Azure compute resources

Labs run in your own Azure subscription and may create billable resources; see LABS.md and delete resources when finished.

Exam or practical application

Separate plan-level requirements (compute, tier, scale) from app-level settings (domain, TLS, deployment slot, backup, access). For private connectivity, identify whether the requirement is inbound to the app or outbound from the app to a backend.

Self-check questions

1. What happens when several apps use the same App Service plan?

   Answer: They share the plan's compute resources; scaling the plan affects the apps hosted on it.

2. Is VNet integration primarily an inbound private endpoint to the app?

   Answer: No. It provides supported outbound connectivity from the app into a virtual network; a private endpoint addresses private inbound access.

3. What should be checked before swapping a staging slot into production?

   Answer: Validate the application and review slot-specific settings, dependencies, and any state or database changes that are not safely reversed by a slot swap.

Sources

- App Service overview
- App Service plans
- Scale up an app in Azure App Service
- Scale instance count manually or automatically
- Configure a custom domain name
- Secure a custom DNS name with a TLS/SSL binding
- Back up an app in Azure
- App Service networking features
- Set up staging environments in Azure App Service

CHAPTER 4

Virtual networking (15–20%)

Virtual network troubleshooting is a process of following the packet path and checking each control in order: address plan, route, security rule, name resolution, service endpoint or private link, and destination health. A correct setting at one layer cannot compensate for a failure at another.

4.1 Virtual networks, subnets, and routing

Learning intent

Create and configure virtual networks and subnets, peer networks, configure public IP addresses and user-defined routes, and troubleshoot connectivity.

Core concepts

An **Azure virtual network (VNet)** is a private network boundary with one or more nonoverlapping IP address spaces. **Subnets** divide that address space for workload placement and policy. Reserve adequate ranges for expected growth and required platform use; Azure reserves addresses in each subnet, so do not plan to consume every address. A subnet is regional and belongs to one VNet.

VNet peering connects VNets using Microsoft's backbone, allowing private IP connectivity subject to routing, security, and address-space requirements. Peering is not automatically transitive: if VNet A peers with B and B peers with C, A does not thereby gain direct connectivity to C. Gateway transit or network appliances can support hub-and-spoke patterns, but must be configured intentionally.

A **public IP address** associates a public endpoint with an Azure resource, subject to the service's configuration and security controls. It is not inherently required for all outbound connectivity and does not replace a firewall. A **route table** contains system routes and optional user-defined routes (UDRs). A UDR can direct traffic to a virtual appliance, virtual network gateway, or another supported next hop. Effective routes show the route selected for a network interface.

How it works, distinctions, limits, and defaults

When troubleshooting, verify that source and destination ranges do not overlap, the resource is in the intended subnet, peering is connected on both sides, and a route exists in both directions. A more specific route can win over a broader route; service routes and platform behavior should be checked in effective routes. NSGs and firewalls may still block permitted routes.

Peering does not itself configure DNS, security rules, or network virtual appliances. A public IP can expose a service to the internet if the attached resource listens and security allows it; prefer private connectivity when public exposure is unnecessary.

Example or worked scenario

A VM in a spoke VNet cannot reach a database in a hub VNet. The administrator verifies address-space overlap, both peering states, gateway or appliance transit design, effective routes on the VM NIC, and NSG rules at the NIC and subnet. A successful ping is not assumed because the service may block ICMP; the actual application port and destination health are tested.

Hands-on activity

In a sandbox, inspect a VNet address plan, subnet ranges, peering status, public IP associations, and effective routes for a network interface. If creating test VNets, choose nonoverlapping ranges and remove them after reviewing connectivity.

Official labs

- Lab 04 – Implement virtual networking
- Lab 05 – Implement intersite connectivity
- Learning path: AZ-104: Configure and manage virtual networks for Azure administrators

Labs run in your own Azure subscription and may create billable resources; see LABS.md and delete resources when finished.

Exam or practical application

Trace connectivity from source to destination and back. Identify the selected route, next hop, and every applicable security boundary. Treat public IP assignment, routing, peering, and authorization as separate parts of the design.

Self-check questions

1. Does peering from A to B and B to C automatically provide transitive connectivity between A and C?

   Answer: No. Peering is not automatically transitive.

2. What does an effective route help an administrator determine?

   Answer: Which route is selected for traffic from a network interface and the associated next hop.

3. If a route permits a packet, is successful application connectivity guaranteed?

   Answer: No. Security rules, DNS, service health, and return routing can still prevent communication.

Sources

- Virtual network overview
- Create, change, or delete a virtual network
- Virtual network peering
- Public IP addresses
- Virtual network traffic routing
- Diagnose a virtual machine routing problem

4.2 Secure access and private connectivity

Learning intent

Create and interpret network security groups (NSGs) and application security groups (ASGs), evaluate effective rules, provide administrative access with Azure Bastion, and select service endpoints or private endpoints for PaaS connectivity.

Core concepts

An **NSG** contains inbound and outbound allow/deny rules evaluated by priority. Rules can use address prefixes, ports, protocol, and service or application security groups. NSGs can be associated with subnets and network interfaces. **Application security groups** let rules refer to workload groups rather than repeatedly listing IP addresses; they are used within supported network designs and do not replace an NSG.

To investigate effective security, inspect rules applied at each association point, rule priority, default rules, and effective security rules on the NIC. A higher-priority matching rule can determine the outcome before a lower-priority rule. NSGs are stateful for established flows, but a new flow still needs a permitted direction and return path.

Azure Bastion provides managed remote connectivity to VMs over TLS through the Azure portal or supported clients, avoiding a public IP on each VM. Bastion itself requires a suitable deployment, subnet, SKU, and network configuration. It reduces exposed management endpoints but does not remove the need to secure identities and VM accounts.

A **service endpoint** extends a VNet subnet's identity to supported Azure services, so a service firewall can allow selected subnets while traffic reaches the service's public endpoint over the Azure backbone. A **private endpoint** assigns a private IP in a VNet to a specific service resource using Azure Private Link. Private endpoints usually require correct private DNS configuration. They are not interchangeable: the endpoint, DNS behavior, supported services, and security requirements differ.

How it works, distinctions, limits, and defaults

If a rule appears correct but traffic fails, check for another NSG association, a higher-priority rule, a route or firewall issue, source address translation, and the destination's own access policy. Network Watcher tools can help identify effective rules and paths.

Service endpoints do not make a service private or remove its public endpoint; firewall configuration controls which networks can access it. Private endpoints do not automatically disable public network access on the service. Disable or restrict the public path separately when required, and confirm name resolution returns the intended private address from the client network.

Example or worked scenario

A VM needs administrative access and must read a storage account without exposing either service broadly. The team uses Bastion for VM management, gives the VM a managed identity with a least-privileged storage data role, and chooses a private endpoint with private DNS when private IP-based service access is required. It separately restricts public network access and validates DNS and effective NSG rules.

Hands-on activity

In a sandbox, inspect a VM's effective security rules and trace why a test port is allowed or denied. Compare the network path and DNS result for a service endpoint and a private endpoint in a test configuration. Verify that a private endpoint has correct DNS records and remove test resources afterward.

Official labs

- Lab 04 – Implement virtual networking
- Learning path: AZ-104: Configure and manage virtual networks for Azure administrators

Labs run in your own Azure subscription and may create billable resources; see LABS.md and delete resources when finished.

Exam or practical application

Match the requirement: rule-based network filtering suggests NSG/ASG; browser-based VM administration without per-VM public IPs suggests Bastion; subnet-based service firewall access suggests service endpoints; a private IP for a specific PaaS resource suggests a private endpoint plus DNS planning.

Self-check questions

1. Does a service endpoint assign a private IP address to the PaaS resource in the VNet?

   Answer: No. A private endpoint provides a private IP in the VNet; a service endpoint extends subnet identity to a supported service.

2. Does adding an NSG allow rule guarantee that traffic can reach the destination?

   Answer: No. Other NSGs, routes, firewalls, DNS, and destination configuration can still block the flow.

3. What additional configuration is commonly needed for private endpoint name resolution?

   Answer: Private DNS configuration that resolves the service name to the private endpoint address from the relevant clients.

Sources

- Network security groups
- Application security groups
- View effective security rules

- What is Azure Bastion?
- Azure virtual network service endpoints
- What is Azure Private Link?
- Azure private endpoint DNS values

4.3 Name resolution and load balancing

Learning intent

Configure Azure DNS, select internal or public load-balancing designs, and troubleshoot name resolution and load distribution.

Core concepts

Azure DNS hosts DNS domains and records. A public DNS zone serves internet-visible records after its name servers are delegated at the domain registrar. A **private DNS zone** resolves names for linked VNets and supports private service resolution when configured with appropriate records and links. DNS records map names to addresses or other names; TTL controls caching duration, not the time required for every client to observe a change.

An **Azure Load Balancer** distributes TCP/UDP flows at layer 4 across healthy backend instances using a rule and health probe. A public load balancer has a public frontend; an internal load balancer uses a private frontend address. Backend pool membership, probe status, rules, NSGs, and return paths all affect results. A probe can mark a backend unavailable even when the VM is running if the application does not respond on the expected port or path.

How it works, distinctions, limits, and defaults

DNS maps a name to an endpoint; it does not ensure the endpoint is reachable or healthy. Verify the resolver used by the client, zone links or delegation, record values, and cache/TTL behavior. Private DNS zone links and private endpoint records must be designed for the client VNets that need resolution.

A load-balancing rule defines frontend/backend ports and protocol; a health probe controls which backends receive new flows. Troubleshoot from the client inward: resolve the name, test the frontend, inspect the rule and backend pool, check probe health, then verify NSGs, routes, and application listeners. Load balancing is not the same as application-layer routing or global disaster recovery.

Example or worked scenario

A two-VM application is reachable by private IP but not by its DNS name. The administrator checks the client resolver, private zone link, record, and TTL before changing the load balancer. Once name resolution works, an internal load-balancer probe remains unhealthy; checking the configured probe port reveals that the service listens on a different port.

Hands-on activity

In a sandbox, inspect a public or private DNS zone, an internal load balancer frontend, backend pool, rule, and probe. Use name lookup from a VM in the intended VNet and compare it with lookup from an unlinked network. Avoid exposing a test backend publicly.

Official labs

- Lab 04 – Implement virtual networking
- Lab 06 – Implement network traffic management
- Learning path: AZ-104: Configure and manage virtual networks for Azure administrators

Labs run in your own Azure subscription and may create billable resources; see LABS.md and delete resources when finished.

Exam or practical application

Distinguish DNS failure from network or application failure. For load balancing, validate the frontend type, port mapping, backend health, probe, and path to each instance before changing unrelated NSGs.

Self-check questions

1. What does a load-balancer health probe control?

   Answer: It determines whether a backend is considered healthy and eligible to receive new traffic under the associated rule.

2. Will creating a private DNS zone automatically make it resolvable from every VNet?

   Answer: No. The relevant VNets and DNS clients must be linked or otherwise configured to use the zone.

3. If DNS resolves to the expected load-balancer address, does that prove an application is healthy?

   Answer: No. Routing, security, load-balancer rules and probes, backend health, and application behavior still need verification.

Sources

- Azure DNS documentation
- Azure DNS private zones
- Azure Load Balancer overview
- Azure Load Balancer health probes
- Troubleshoot Azure Load Balancer

CHAPTER 5

Monitoring and recovery (10–15%)

Monitoring helps administrators detect conditions; recovery services help restore operations after data loss or a larger outage. Neither works well without a defined target: what must be observed, what threshold matters, who responds, and how quickly the workload must recover.

5.1 Azure Monitor and Network Watcher

Learning intent

Interpret Azure Monitor metrics, configure diagnostic logs, query and analyze logs, create alerts and action groups, use Azure Monitor Insights, and diagnose network paths with Network Watcher and Connection Monitor.

Core concepts

Azure Monitor collects and analyzes telemetry. **Metrics** are numeric time-series measurements with dimensions, such as CPU percentage or request count. They are useful for trends and near-real-time alerting. **Logs** are records stored in a Log Analytics workspace or another supported destination; they support querying and correlation but require appropriate diagnostic settings and retention configuration. Resource diagnostic settings route selected logs and metrics to destinations; they do not enable every resource log automatically.

Log queries use **Kusto Query Language (KQL)** to filter, aggregate, join, and summarize records. Start with a time range, identify the correct table, filter early, and validate the units and dimensions before drawing conclusions. **Alert rules** evaluate metrics, logs, or activity conditions. An **action group** describes notification or automation actions; an **alert processing rule** can alter or suppress alert handling under specified conditions, such as a maintenance period. Separate detection logic from notification and response ownership.

Azure Monitor Insights provides curated monitoring views for supported resources such as VMs, storage accounts, and networks. It can surface relevant metrics and dependencies but does not eliminate the need to configure data collection, permissions, or alerting. **Network Watcher** offers diagnostics such as IP flow verify, next hop, connection troubleshoot, packet capture, and topology. **Connection Monitor** tests connectivity over time between sources and destinations and can alert on loss or latency.

How it works, distinctions, limits, and defaults

Metrics and logs answer different questions; availability, granularity, retention, and cost vary by metric, log category, resource, and destination. Confirm that collection is enabled before relying on a chart. A metric anomaly is evidence to investigate, not proof of a root cause.

Alerts require more than a threshold: scope, condition, evaluation frequency, severity, action group, and operational ownership all matter. Avoid noisy thresholds and verify that an action group reaches a monitored contact. Network diagnostics should follow the actual source, destination, protocol, port, DNS, and route.

Example or worked scenario

Users report intermittent slow responses from a web app. The operator checks request and server metrics, queries relevant logs over the incident window, and correlates a latency increase with a dependent service. An alert rule detects sustained latency and invokes an action group. Connection Monitor then identifies loss on a path to a private backend; Network Watcher tools help inspect route and filtering state.

Hands-on activity

In a test resource group, inspect available metrics and diagnostic categories for a VM or storage account. Route a small set of logs to a test workspace, run a time-bounded KQL query, and create a test metric or log alert with a nonproduction action group. In a sandbox VNet, inspect Network Watcher connectivity tools and remove test settings after review.

Official labs

- Lab 11 – Implement monitoring
- Lab 05 – Implement intersite connectivity
- Learning path: AZ-104: Monitor and back up Azure resources

Labs run in your own Azure subscription and may create billable resources; see LABS.md and delete resources when finished.

Exam or practical application

For each symptom, choose the telemetry that can observe it: metrics for numeric trends, logs for event/detail analysis, and network tools for reachability and path behavior. Trace alert evaluation through condition, scope, action group, and response procedure.

Self-check questions

1. What is a common distinction between an Azure Monitor metric and a log?

   Answer: Metrics are numeric time-series measurements; logs are records that can be queried and correlated for detailed analysis.

2. Does creating an alert rule automatically notify an administrator?

   Answer: No. The rule must be configured with an action group or another response mechanism.

3. Which tool is suited to monitoring connectivity and latency between endpoints over time?

   Answer: Connection Monitor.

Sources

- Azure Monitor overview
- Metrics in Azure Monitor
- Azure Monitor diagnostic settings
- Log Analytics overview
- Kusto Query Language (KQL) overview
- Azure Monitor alerts overview
- Create and manage action groups
- Alert processing rules
- Azure Monitor Insights overview
- What is Azure Network Watcher?
- Monitor network connectivity with Connection Monitor

5.2 Backup and disaster recovery

Learning intent

Create the appropriate vault, configure and operate Azure Backup policies, restore protected data, configure Azure Site Recovery (ASR), perform a failover, and interpret backup reports and alerts.

Core concepts

Azure Backup protects supported workloads using a backup policy that defines when recovery points are created and how long they are retained. A **Recovery Services vault** stores backup and Site Recovery configuration for supported workloads; an **Azure Backup vault** supports backup for other supported data sources and newer scenarios. Select the vault type from the workload's current support matrix rather than assuming the two are interchangeable.

A backup policy describes schedule and retention, subject to workload and vault capabilities. After enabling backup, monitor job status, retention, and alerts. A successful backup is not proven recoverable until a restore is tested. Restore choices can include restoring files, disks, or a VM, depending on the protected workload. Plan for identity, networking, keys, permissions, and application consistency during recovery.

Azure Site Recovery replicates supported workloads to another location or region and orchestrates recovery during a disruption. Replication and failover are distinct from backup: replication can reproduce unwanted changes or corruption, while backup provides retained recovery points. A **test failover** validates a recovery plan in an isolated network; a planned or unplanned failover changes the serving location. Commit and reprotect steps depend on the scenario and should be understood before production use.

Backup reports and alerts expose job health, protected-item coverage, and issues requiring attention. Configure reporting and notification paths, review failures, and assign an owner for remediation. Retention, data residency, network access, and costs are part of the protection design.

How it works, distinctions, limits, and defaults

Recovery objectives should be explicit: **recovery point objective (RPO)** is the acceptable data-loss window; **recovery time objective (RTO)** is the acceptable restoration time. Backup frequency and replication behavior affect RPO; restore throughput, dependencies, and tested automation affect RTO. Service defaults do not automatically meet business objectives.

Vault redundancy, soft delete, immutability, encryption, private connectivity, and role separation can strengthen protection, but feature availability differs by workload and configuration. Prevent backup operators from having unnecessary delete or production privileges. Validate the actual restore or failover path and record the result.

Example or worked scenario

A VM must recover from accidental deletion and from a regional outage. The team protects it with Azure Backup in a supported vault and tests restoring a VM or disks. It separately configures Site Recovery for regional continuity, defines a recovery plan and isolated test network, and performs a test failover. A quarterly review confirms backup alerts reach the on-call team and measured recovery times meet objectives.

Hands-on activity

In a nonproduction subscription, inspect supported vault types for a test workload, create a short-retention backup policy, and enable protection. Monitor the first job and perform a documented test restore where possible. Separately review ASR prerequisites and perform only a test failover in an isolated environment. Clean up test replicas and vault resources according to product guidance.

Official labs

- Lab 10 – Implement data protection
- Learning path: AZ-104: Monitor and back up Azure resources

Labs run in your own Azure subscription and may create billable resources; see LABS.md and delete resources when finished.

Exam or practical application

Use backup when the requirement is retained point-in-time recovery; use Site Recovery when the requirement is orchestrated continuity at a secondary location. Identify workload support, vault type, policy, recovery objective, and verification/reporting need. A failover plan should include dependencies and post-failover steps, not just the compute resource.

Self-check questions

1. What is the main difference between Azure Backup and Site Recovery?

   Answer: Backup retains recovery points for restore; Site Recovery replicates workloads and orchestrates failover to support continuity.

2. How do RPO and RTO differ?

Answer: RPO describes acceptable data loss measured as time; RTO describes acceptable time to restore service.

3. Does a successful backup job prove that the application can be recovered within its RTO?

   Answer: No. Recovery must be tested, including application dependencies and measured restoration time.

Sources

- Azure Backup documentation
- Azure Backup vault overview
- Recovery Services vault overview
- Azure Backup support matrix
- Back up an Azure VM
- Restore an Azure VM
- Azure Backup reports
- Azure Site Recovery overview
- Run a test failover in Azure Site Recovery
- Fail over and fail back virtual machines